

```

import asyncio
import logging
import os
from datetime import datetime
import requests
from telegram import Bot
from telegram.ext import Application, CommandHandler, ContextTypes
from apscheduler.schedulers.asyncio import AsyncIOScheduler

# ===== НАСТРОЙКИ (из переменных окружения Railway) =====
TELEGRAM_TOKEN = os.environ["TELEGRAM_TOKEN"]
OPENWEATHER_API_KEY = os.environ["OPENWEATHER_API_KEY"]
CHAT_ID = os.environ["CHAT_ID"]

# Координаты Крутиц, Шиловский район, Рязанская область
LAT = 54.5167
LON = 40.9333

# Время отправки сообщений (по UTC, Москва = UTC+3)
MORNING_HOUR = 6      # 09:00 MCK
MORNING_MIN = 0
EVENING_HOUR = 14     # 17:00 MCK
EVENING_MIN = 0
# =====

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def get_weather_data():
    """Получает данные погоды с OpenWeatherMap."""
    url = "https://api.openweathermap.org/data/2.5/forecast"
    params = {
        "lat": LAT,
        "lon": LON,
        "appid": OPENWEATHER_API_KEY,
        "units": "metric",
        "lang": "ru",
        "cnt": 8 # Прогноз на 24 часа (шаг 3 часа)
    }
    response = requests.get(url, params=params, timeout=10)
    response.raise_for_status()
    return response.json()

```

```

def calculate_skydive_probability(forecast_list):
    """
    Рассчитывает вероятность благоприятной погоды для прыжков.
    Учитывает: облачность, осадки, скорость ветра, видимость.
    """
    scores = []

    for item in forecast_list[:4]: # Ближайшие 12 часов
        score = 100

        # Облачность (0-100%)
        clouds = item["clouds"]["all"]
        if clouds > 80:
            score -= 40
        elif clouds > 50:
            score -= 20
        elif clouds > 30:
            score -= 10

        # Осадки
        if "rain" in item or "snow" in item:
            score -= 50
        weather_id = item["weather"][0]["id"]
        if weather_id < 700: # Гроза, дождь, снег
            score -= 40

        # Скорость ветра (м/с)
        wind_speed = item["wind"]["speed"]
        if wind_speed > 10:
            score -= 40
        elif wind_speed > 7:
            score -= 25
        elif wind_speed > 5:
            score -= 10

        # Порывы ветра
        wind_gust = item["wind"].get("gust", 0)
        if wind_gust > 12:
            score -= 20
        elif wind_gust > 8:
            score -= 10

        # Видимость (если есть)
        visibility = item.get("visibility", 10000)
        if visibility < 3000:
            score -= 30

```

```

        elif visibility < 5000:
            score -= 15

        scores.append(max(0, score))

    return round(sum(scores) / len(scores)) if scores else 0

def get_probability_emoji(prob):
    if prob >= 80:
        return "🟢"
    elif prob >= 60:
        return "🟡"
    elif prob >= 40:
        return "🟠"
    else:
        return "🔴"

def get_wind_direction(deg):
    directions = ["С", "СВ", "В", "ЮВ", "Ю", "ЮЗ", "З", "СЗ"]
    return directions[round(deg / 45) % 8]

def format_weather_message(data, period_label):
    """Формирует итоговое сообщение для Telegram."""
    forecast_list = data["list"]
    probability = calculate_skydive_probability(forecast_list)
    emoji = get_probability_emoji(probability)

    # Текущие условия
    current = forecast_list[0]
    temp = round(current["main"]["temp"])
    feels_like = round(current["main"]["feels_like"])
    wind_speed = current["wind"]["speed"]
    wind_gust = current["wind"].get("gust", wind_speed)
    wind_dir = get_wind_direction(current["wind"]["deg"])
    clouds = current["clouds"]["all"]
    humidity = current["main"]["humidity"]
    visibility = current.get("visibility", 10000) // 1000
    desc = current["weather"][0]["description"].capitalize()

    # Сводка по ближайшим 12 часам
    summary_lines = []
    for item in forecast_list[:4]:
        time_str = datetime.utcfromtimestamp(item["dt"]).strftime("%H:%M")
        t = round(item["main"]["temp"])

```

```

        w = item["wind"]["speed"]
        c = item["clouds"]["all"]
        d = item["weather"][0]["description"]
        summary_lines.append(f"    {time_str} UTC | {t}°C | 🌡️ {w} м/с | ☁️ {c}% |

summary = "\n".join(summary_lines)

message = (
    f"🪂 *Прогноз для прыжков – {period_label}*\n"
    f"📍 Крутицы, Рязанская обл.\n\n"
    f"{emoji} *Вероятность благоприятной погоды: {probability}%*\n\n"
    f"☀️ *Сейчас:* {desc}\n"
    f"🌡️ Температура: {temp}°C (ощущается {feels_like}°C)\n"
    f"💨 Ветер: {wind_speed} м/с ({wind_dir}), порывы до {wind_gust:.1f} м/с\
    f"☁️ Облачность: {clouds}%\n"
    f"💧 Влажность: {humidity}%\n"
    f"👁️ Видимость: {visibility} км\n\n"
    f"🕒 *Ближайшие 12 часов (UTC):*\n"
    f"{summary}\n\n"
    f"_{{datetime.utcnow().strftime('%d.%m.%Y %H:%M')}} UTC_"
)

return message

async def send_forecast(bot: Bot, period_label: str):
    """Получает погоду и отправляет сообщение."""
    try:
        data = get_weather_data()
        message = format_weather_message(data, period_label)
        await bot.send_message(
            chat_id=CHAT_ID,
            text=message,
            parse_mode="Markdown"
        )
        logger.info(f"Прогноз отправлен: {period_label}")
    except Exception as e:
        logger.error(f"Ошибка при отправке прогноза: {e}")
        await bot.send_message(
            chat_id=CHAT_ID,
            text=f"⚠️ Не удалось получить прогноз погоды: {e}"
        )

async def cmd_start(update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        f"🪂 *Бот прогноза для парашютистов*\n\n"

```

```

        "Я буду присылать прогноз дважды в день:\n"
        "🌅 Утром (~09:00 МСК)\n"
        "🌃 Вечером (~17:00 МСК)\n\n"
        "Команды:\n"
        "/weather — прогноз прямо сейчас",
        parse_mode="Markdown"
    )

async def cmd_weather(update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("⌚ Загружаю прогноз...")
    try:
        data = get_weather_data()
        message = format_weather_message(data, "сейчас")
        await update.message.reply_text(message, parse_mode="Markdown")
    except Exception as e:
        await update.message.reply_text(f"⚠️ Ошибка: {e}")

def main():
    app = Application.builder().token(TELEGRAM_TOKEN).build()

    # Команды
    app.add_handler(CommandHandler("start", cmd_start))
    app.add_handler(CommandHandler("weather", cmd_weather))

    # Планировщик
    scheduler = AsyncIOScheduler()

    scheduler.add_job(
        lambda: asyncio.create_task(send_forecast(app.bot, "утро 🌅")),
        "cron", hour=MORNING_HOUR, minute=MORNING_MIN
    )
    scheduler.add_job(
        lambda: asyncio.create_task(send_forecast(app.bot, "вечер 🌃")),
        "cron", hour=EVENING_HOUR, minute=EVENING_MIN
    )

    scheduler.start()
    logger.info("Бот запущен. Планировщик активен.")

    app.run_polling()

if __name__ == "__main__":
    main()

```

